CS5296 Group 5

Fresh Local Rerun Report: HPA vs KEDA

Branch: rerun-20260423-fresh-artifacts
Environment: macOS host + Rancher Desktop K3s (4 CPU / 8 GiB VM)

Abstract

This report summarizes a clean rerun of the repository on a new Git branch and clean worktree. The rerun rebuilt the consumer image, redeployed RabbitMQ plus both autoscaling strategies, collected six new trial traces, regenerated the figures, and packaged a fresh PDF report. In this local rerun, the HPA baseline reacted faster on average (12.66s) than KEDA (22.45s), while both strategies still reached the configured peak of 10 pods. The result differs from the original project hypothesis and is likely sensitive to the local runtime, KEDA polling path, and the producer's achieved publish rate.

Keywords: Kubernetes autoscaling, RabbitMQ, HorizontalPodAutoscaler, KEDA, reproducibility

1. Introduction And Background

The repository compares two autoscaling strategies for the same message-processing workload: a baseline HorizontalPodAutoscaler that follows CPU pressure and a KEDA deployment that follows RabbitMQ backlog. The original project hypothesis expected queue-aware scaling to react earlier because backlog exists before CPU utilization saturates inside the consumer pods.

This branch reruns the experiment from scratch rather than reusing existing outputs. The goal is branch-level reproducibility: a clean checkout, a fresh cluster, six new trial traces, freshly rendered figures, and a report that is generated only from the new CSV files committed on this branch.

The rerun also acts as a validation pass on portability. Running on an Apple Silicon macOS host surfaced several issues that were not obvious from the original AWS-focused runbook, including container base-image compatibility, BSD timestamp behavior, and the need to separate the active burst from the post-burst observation window.

2. System Design

Both groups share the same core data path: a Python producer sends messages to RabbitMQ, a Spring Boot consumer processes them, and a metrics collector samples pod counts plus queue depth into CSV. The only variable between groups is the autoscaling controller.

Dimension	Baseline (HPA)	KEDA
Signal source	CPU utilization from metrics-server	RabbitMQ queue depth via KEDA trigger
Decision loop	HorizontalPodAutoscaler controller	KEDA operator and ScaledObject
Min / max replicas	1 / 10	1 / 10
Expected strength	Stable and built-in	Closer to business backlog
Expected risk	Can react after CPU is already hot	Depends on polling interval and trigger path

To make the rerun stable on this machine, the branch introduces an arm64-compatible consumer runtime image, `python3` defaults in the experiment wrapper, portable timestamp generation in the collector, request timeouts for management queries, and a dedicated burst pattern file without a built-in idle tail.

3. Experimental Setup

Each trial starts from a clean queue and a single consumer replica. The producer warms up the broker for 10 seconds at a low rate, then injects a 10 second burst with a target 1000 msg/s rate. The actual publish rate is lower because the current producer publishes synchronously through `pika`, but the pattern remains identical across both groups.

Item	Value
Branch	rerun-20260423-fresh-artifacts
Cluster runtime	Rancher Desktop 1.22 with local K3s
VM sizing	4 vCPU / 8 GiB RAM
Broker	RabbitMQ 3.12 management image
Consumer image	cs5296-consumer:rerun-20260423-local
Burst pattern	10 s warm-up at 5 msg/s, then 10 s active burst targeting 1000 msg/s
Observation tail	180 s after publishing
Trial count	6 total runs: baseline x3 and keda x3

Metric	Definition used in this rerun
Reaction latency	Seconds from burst start to the first observed scale-up event.
Scale-up time	Seconds from burst start until the group reached its peak pod count.
Drain time	Seconds from burst start until the queue returned to zero backlog.
Average throughput	Delivered messages divided by the active send window used by the analysis script.
Messages delivered	Messages observed as delivered by the end of the collected trace.

4. Results Summary

The comparison table below aggregates the six fresh trials. Lower reaction latency and drain time are better, while higher throughput is better. Peak pods reached the configured ceiling in both groups, so the meaningful differences came from controller timing rather than maximum scale.

Metric	Baseline mean +/- std	KEDA mean +/- std	Note
Reaction latency (s)	12.66 +/- 5.24	22.45 +/- 2.97	lower is better
Scale-up time (s)	46.54 +/- 4.64	48.26 +/- 2.87	lower is better
Drain time (s)	55.10 +/- 17.53	70.38 +/- 46.28	lower is better
Peak pods	10.00 +/- 0.00	10.00 +/- 0.00	same max for both groups
Avg throughput (msg/s)	25.46 +/- 8.48	22.76 +/- 10.93	higher is better
Messages delivered	3236.67 +/- 2371.77	5537.33 +/- 1286.26	context only

- HPA reacted faster on average: 12.66s versus 22.45s.

- HPA drained the queue faster on average: 55.10s versus 70.38s.

- Average throughput also favored HPA in this rerun: 25.46 msg/s versus 22.76 msg/s.

- KEDA remained more variable across runs, suggesting the local queue-metric polling path was less steady than expected.

- The fresh rerun therefore does not reproduce the original narrative that KEDA is strictly faster in this workload.

This outcome should be interpreted honestly: the branch proves that the experiment can be rerun end-to-end from scratch, but it also shows the result is sensitive to environment choice, achieved burst rate, and instrumentation path. See `docs/ARTIFACT.md` for the exact reproduction procedure and branch-specific notes.

Figure 1. Pod scaling timeline

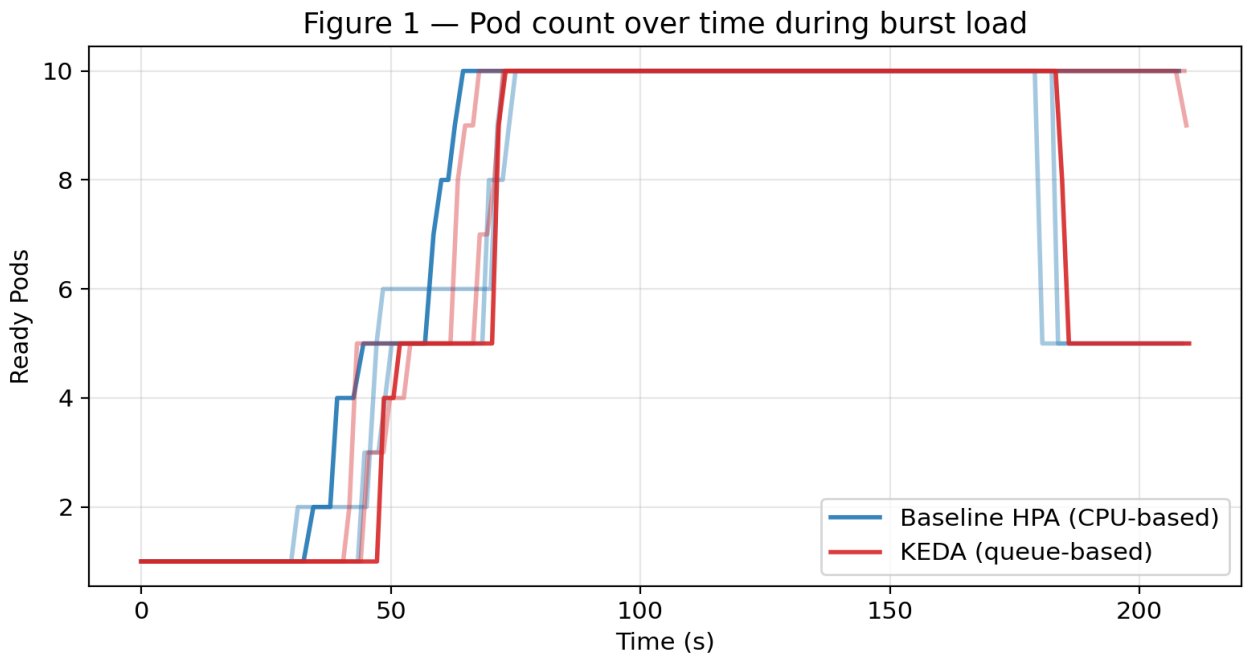

fig1-pod-scaling-timeline.png

Figure 2. Queue depth timeline

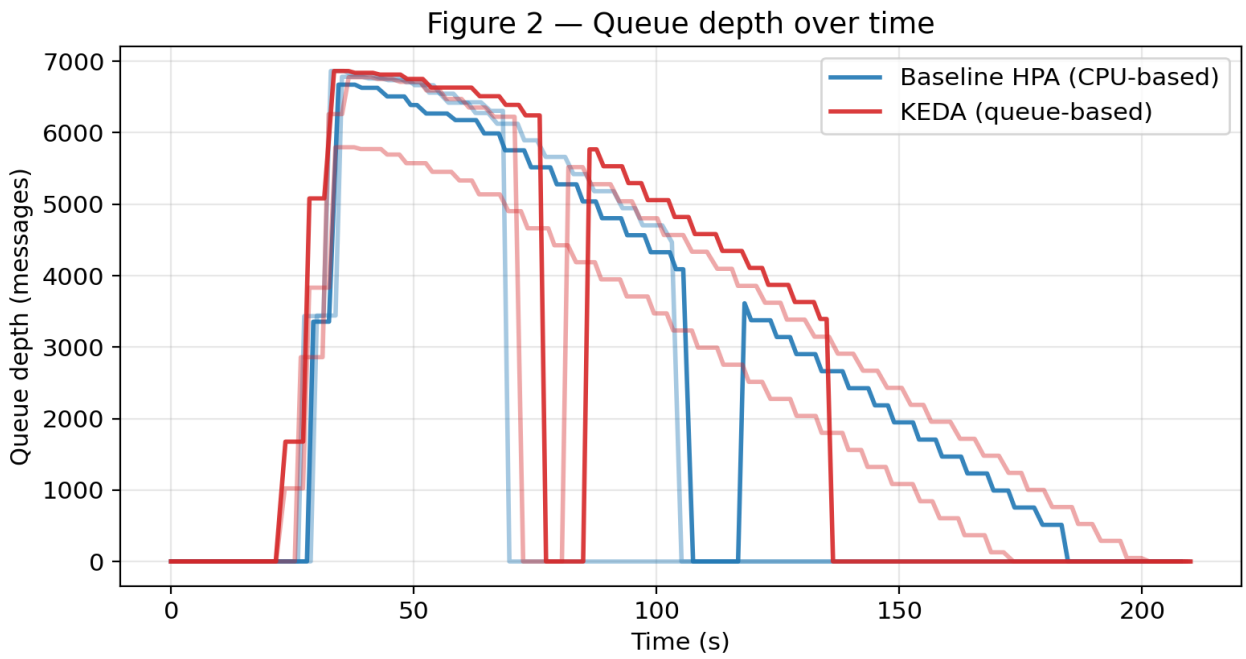

fig2-queue-depth-timeline.png

Figure 3. Reaction latency comparison

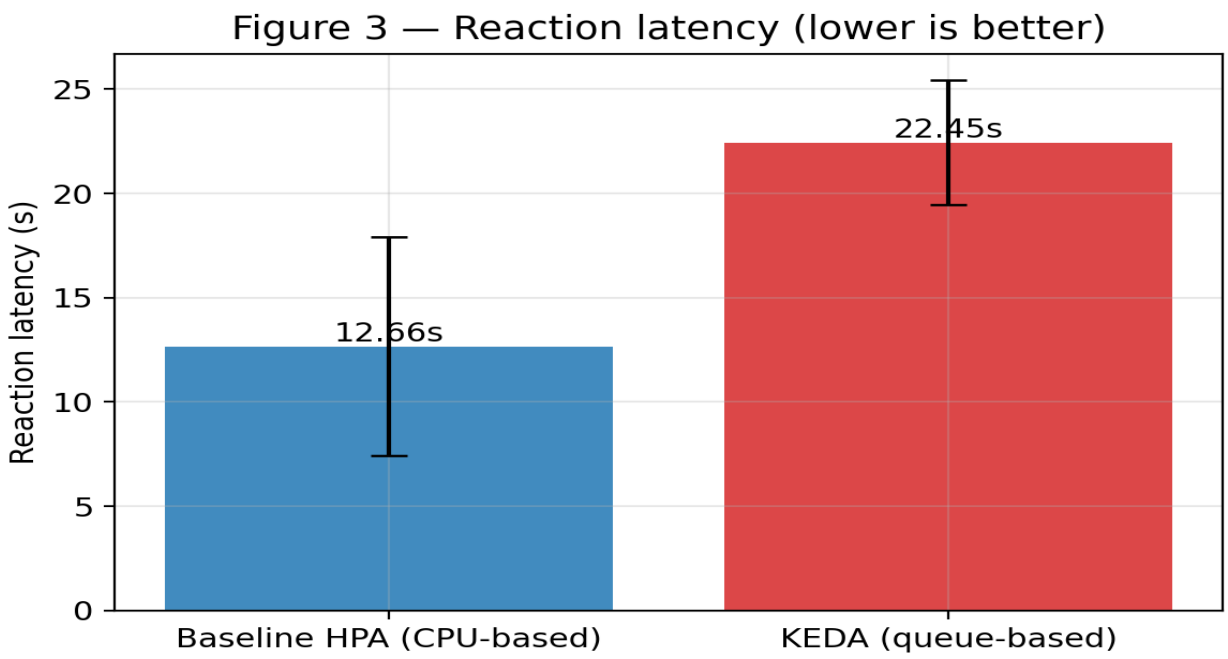

fig3-reaction-latency-bar.png

Figure 4. Throughput comparison

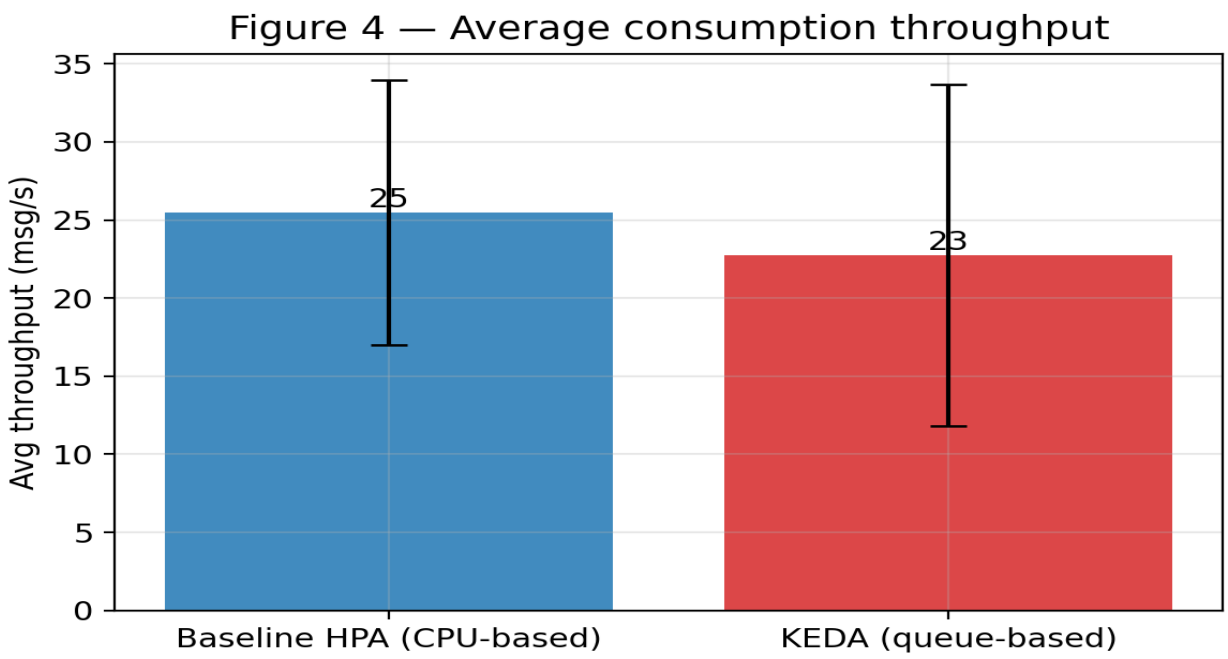

fig4-throughput-comparison.png